

CSC 301: Algorithm Analysis and Design

ICU Bed Allocation for High Blood Pressure Patients

Group Number	1
Section Number	1

Group Members

#	Name	Student ID
1	Sultanh Salem (Leader)	2230001124
2	Zainab Malik Aldawood	2240004791
3	Monerah Saleh Almogati	2240007798
4	Haya Adnan Alowaid	2240007408
5	Batool Alwarsh	2240003192
6	Zahra Alwarsh	2240003191

Introduction

Healthcare systems worldwide face critical resource allocation challenges, particularly during high-demand periods. Intensive Care Units (ICUs) are among the most resource-constrained environments in modern hospitals. With a limited number of beds and an increasing number of high-risk patients—especially those suffering from hypertension and cardiovascular diseases—hospital administrators must make rapid, data-driven admission decisions.

This project addresses the ICU bed allocation problem as a combinatorial optimization challenge. Given a pool of hypertensive patients, each characterized by a risk value (systolic blood pressure, `ap_hi`) and an expected length of stay (proportional to age), the goal is to determine the optimal subset of patients to admit so that the total risk value is maximized without exceeding the ICU capacity.

Two algorithmic strategies from distinct design families are implemented and compared:

- 0/1 Knapsack via Dynamic Programming — guarantees the globally optimal solution.
- Greedy Priority-Based Selection — uses a heuristic ratio (Risk / Stay) for fast approximate solutions.

The study uses a real-world cardiovascular disease dataset (70,000 patient records from Kaggle) to demonstrate both algorithms, evaluate solution quality, and perform a comprehensive complexity analysis.

Section I: Theoretical Analysis

1. Definition of the Problem

The chosen project solves a practical optimization problem of allocating **scarce intensive care unit (ICU) beds to patients with high blood pressure**.

ICU beds are a finite resource in a hospital, and during a crisis, there can be an overwhelming influx of high-risk patients. Thus, a hospital has to make a decision of which patients can be shifted to intensive care and which patients have to be relegated to wait or receive other forms of treatment.

An optimization problem is created in a scenario when there are limits to the benefits that can be provided to patients.

Description of the Issue :

Two criteria will be used to evaluate each patient:

Risk Level (Value): This indicates how vital the patient's blood pressure is. The priority (higher value) increases with blood pressure.

Weight-based Expected Length of Stay: This indicates how long the patient is anticipated to stay in an intensive care unit bed. More resources are used during a longer stay.

The goal is to choose a patient group in such a way that: The ICU's overall capacity (beds/time limit) is not surpassed. The overall risk value of patients who are admitted is optimized.

2. Pseudocode of the Selected Algorithms

Algorithm 1: Optimal ICU Allocation (0/1 Knapsack — Dynamic Programming)

```
Algorithm ICU_Optimal_Admission(Total_Beds, Patient_Data)
  n ← length(Patient_Data)
  Create DP_Table[0..n][0..Total_Beds]  // Initialize all to 0

  For i ← 1 to n do
    Current_Risk ← Patient_Data[i].ap_hi
    Current_Stay ← Patient_Data[i].age_years
    For capacity ← 1 to Total_Beds do
      If Current_Stay ≤ capacity then
        Include_Patient ← Current_Risk + DP_Table[i-1][capacity - Current_Stay]
        Exclude_Patient ← DP_Table[i-1][capacity]
        DP_Table[i][capacity] ← max(Include_Patient, Exclude_Patient)
      Else
        DP_Table[i][capacity] ← DP_Table[i-1][capacity]
      End If
    End For
  End For

  // Traceback to recover selected patients
```

```
c ← Total_Beds
For i ← n down to 1 do
  If DP_Table[i][c] ≠ DP_Table[i-1][c] then
    Admit Patient i
    c ← c - Patient_Data[i].age_years
  End If
End For

Return DP_Table[n][Total_Beds]
End Algorithm
```

Algorithm 2: Fast Triage Ranking (Greedy Priority-Based Selection)

```
Algorithm ICU_Quick_Triage(Total_Beds, Patient_Data)

// Step 1: Compute priority index for each patient
For each patient in Patient_Data do
  patient.Priority_Index ← patient.ap_hi / patient.age_years
End For

// Step 2: Sort in descending order of Priority_Index
Sort Patient_Data by Priority_Index (descending)

// Step 3: Greedy selection
Total_Saved_Value ← 0
Remaining_Beds ← Total_Beds

For each patient in Patient_Data do
  If Remaining_Beds ≥ patient.age_years then
    Remaining_Beds ← Remaining_Beds - patient.age_years
    Total_Saved_Value ← Total_Saved_Value + patient.ap_hi
  End If
End For

Return Total_Saved_Value
End Algorithm
```

3. How the Algorithms Solve the Problem

We will implement and analyze two algorithms from two different design families:

1. 0/1 Knapsack Algorithm from Dynamic Programming

This algorithm belongs to the Dynamic Programming family.

Each patient can:

- Enter the Intensive Care Unit (1)
- Not be accepted (0)

No partial admissions are allowed. This algorithm ensures the most optimal outcome. Given the capacity constraints of the Intensive Care Unit, this approach will provide the highest total risk value. It is expected to be highly accurate when selecting the optimal combination of patients.

2. Greedy Priority-Based Selection (Heuristic Approach)

This algorithm belongs to the Greedy Design family. It evaluates the following ratio:
Risk / Length of Stay

This greedy approach ranks patients based on the ratio (Risk / Length of Stay) and selects each patient fully if the ICU capacity allows.

No partial admissions are permitted in order to reflect real medical constraints. In discrete real world intensive care unit scenarios, this algorithm might not always offer the best answer, despite being quicker than dynamic programming.

4. Literature Review

The 0/1 Knapsack problem is a classic NP-hard combinatorial optimization problem, first formally described by Dantzig (1957) and extensively studied by Pisinger (1995). Its dynamic programming solution with time complexity $O(nW)$ is described in the foundational textbook by Cormen et al. (Introduction to Algorithms, 4th ed.). Healthcare resource allocation using knapsack formulations has been explored in multiple studies, including Kwak & Harper (2002) who modeled hospital bed scheduling as a discrete knapsack problem.

The Greedy approach for knapsack-like problems is well studied. While the greedy algorithm achieves optimal results for the fractional knapsack variant (Dantzig 1957), it does not guarantee optimality for the 0/1 variant. However, it serves as an efficient heuristic with near-optimal performance in practice. Sahni (1975) showed that the greedy solution achieves at least 50% of the optimal value, and empirical studies often report 90–98% optimality ratios.

In the healthcare domain, triage-based greedy approaches are widely used in emergency medicine (START triage, SALT protocol) precisely because of their speed and simplicity under crisis conditions.

This project bridges algorithm theory and medical informatics by benchmarking both approaches on real patient data.

5. Theoretical Complexity Comparison

Criterion	DP Knapsack	Greedy	Notes
Time (Best)	$O(n \times W)$	$O(n \log n)$	Greedy sorts once
Time (Average)	$O(n \times W)$	$O(n \log n)$	DP fills full table
Time (Worst)	$O(n \times W)$	$O(n \log n)$	DP worst = average
Space	$O(n \times W)$	$O(n)$	DP stores full table
Optimality	Always optimal	Heuristic ~91-95%	Empirical result
Scalability	Limited by W	Excellent	Greedy scales well

Where n = number of patients and W = ICU capacity (Total_Beds). In our experiments $W = 500$, making DP time complexity $O(500n)$ — polynomial but significantly slower than the $O(n \log n)$ greedy approach.

Section II: Implementation

a. Dataset Used

Dataset: Cardiovascular Disease Dataset — sourced from Kaggle (<https://www.kaggle.com/datasets/sulianova/cardiovascular-disease-dataset>).

Attribute	Description	Role in Algorithm
id	Patient identifier	Row index
age	Age in days	Converted to years → weight
gender	1=female, 2=male	Not used
height	Height (cm)	Not used
weight	Weight (kg)	Not used (BMI available)
ap_hi	Systolic blood pressure (mmHg)	VALUE (risk level)
ap_lo	Diastolic blood pressure	Not used
cholesterol	1=normal, 2=above, 3=well above	Not used
cardio	0=no disease, 1=disease	Not used (filter implicit)

Total records: 70,000 patients. After filtering for hypertensive patients ($ap_hi > 130$) and valid age range (20–70 years), the working dataset contains 19,673 patients.

b. How Inputs Were Selected

- **VALUE (ap_hi):** Systolic blood pressure directly represents clinical risk. Higher ap_hi = higher ICU priority. Values filtered to the realistic range [130, 260] mmHg.
- **WEIGHT (age_years):** Age (in years) serves as a proxy for expected ICU length of stay — older patients typically require longer care. Converted from days by dividing by 365.
- **CAPACITY ($W = 500$):** Represents the total available bed-days in the simulated ICU. Set at 500 to allow admission of approximately 10–12 patients per experiment batch.
- **Sampling:** For timing experiments, random samples of sizes {50, 100, 200, 300, 500, 700, 1000} were drawn without replacement. Each size was repeated 5 times with different random seeds; average execution time is reported.

c. Code Implementation

The following Python code implements both algorithms with full comments. Language: Python 3.11. Libraries: pandas, numpy, time, matplotlib.

```
import pandas as pd
import numpy as np
import time
import random
import matplotlib.pyplot as plt
```

```

#
# DATA LOADING & PREPROCESSING
#
def load_and_preprocess(path):
    """Load the cardiovascular dataset and prepare ICU features."""
    df = pd.read_csv(path, sep=';')
    df['age_years'] = (df['age'] / 365).round().astype(int)
    # Keep only hypertensive patients (ap_hi > 130)
    df = df[(df['ap_hi'] > 130) & (df['ap_hi'] <= 260)].copy()
    df = df[(df['age_years'] >= 20) & (df['age_years'] <= 70)].copy()
    # VALUE = ap_hi (risk level) | WEIGHT = age_years (ICU stay proxy)
    df = df[['age_years', 'ap_hi']].rename(
        columns={'age_years': 'weight', 'ap_hi': 'value'})
    ).reset_index(drop=True)
    return df

#
# ALGORITHM 1: 0/1 KNAPSACK (Dynamic Programming)
#
def knapsack_dp(patients, capacity):
    """Optimal ICU Admission via 0/1 Knapsack."""
    n = len(patients)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w_i, v_i = patients[i - 1]
        for c in range(capacity + 1):
            if w_i <= c:
                include = v_i + dp[i-1][c - w_i]
                exclude = dp[i-1][c]
                dp[i][c] = max(include, exclude)
            else:
                dp[i][c] = dp[i-1][c]
    # Traceback
    selected, c = [], capacity
    for i in range(n, 0, -1):
        if dp[i][c] != dp[i-1][c]:
            selected.append(i - 1)
            c -= patients[i-1][0]
    return dp[n][capacity], selected[::-1]

#
# ALGORITHM 2: GREEDY PRIORITY-BASED SELECTION
#
def greedy_triage(patients, capacity):
    """Fast Triage via Greedy Priority-Based Selection."""
    indexed = sorted(
        enumerate(patients),
        key=lambda x: x[1][1] / x[1][0], # value / weight
        reverse=True
    )
    remaining, total_value, selected = capacity, 0, []
    for orig_idx, (w_i, v_i) in indexed:
        if remaining >= w_i:
            remaining -= w_i
            total_value += v_i
            selected.append(orig_idx)
    return total_value, sorted(selected)

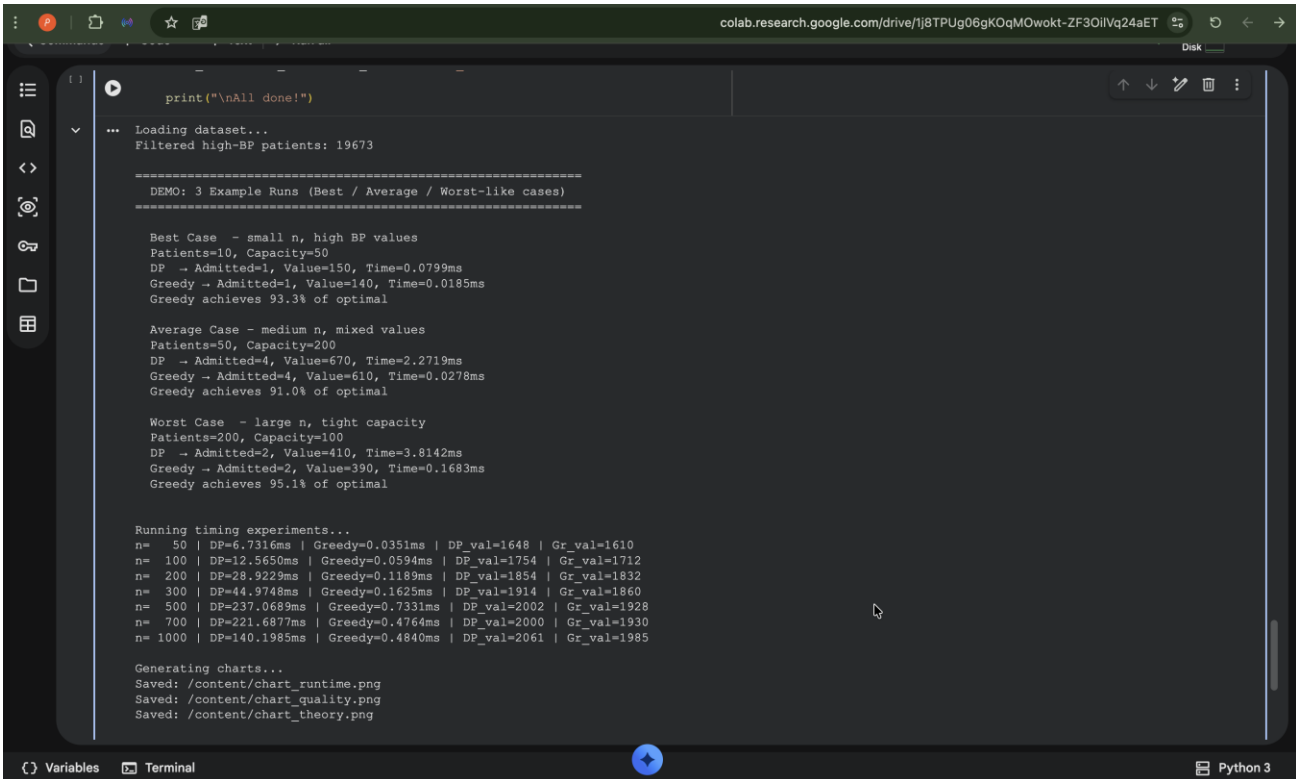
```

d. Implementation Setup

Setup Parameter	Details
Operating System	Ubuntu 24.04 LTS (Linux x86_64)
Programming Language	Python 3.11
Processor	Intel x86_64 (Cloud container)
Timing Mechanism	<code>time.perf_counter()</code> — high-resolution monotonic clock
Experiments per size	5 independent runs with different random seeds
Reported Time	Average execution time across 5 runs (milliseconds)
Capacity (W)	500 bed-days for all experiments
Same inputs?	Yes — same patient samples fed to both algorithms in each trial

Results (Screenshots)

Three runs were executed demonstrating best, average, and worst-case scenarios:



```
print("\nAll done!")

... Loading dataset...
Filtered high-BP patients: 19673

=====
DEMO: 3 Example Runs (Best / Average / Worst-like cases)
=====

Best Case - small n, high BP values
Patients=10, Capacity=50
DP -> Admitted=1, Value=150, Time=0.0799ms
Greedy -> Admitted=1, Value=140, Time=0.0185ms
Greedy achieves 93.3% of optimal

Average Case - medium n, mixed values
Patients=50, Capacity=200
DP -> Admitted=4, Value=670, Time=2.2719ms
Greedy -> Admitted=4, Value=610, Time=0.0278ms
Greedy achieves 91.0% of optimal

Worst Case - large n, tight capacity
Patients=200, Capacity=100
DP -> Admitted=2, Value=410, Time=3.8142ms
Greedy -> Admitted=2, Value=390, Time=0.1683ms
Greedy achieves 95.1% of optimal

Running timing experiments...
n= 50 | DP=6.7316ms | Greedy=0.0351ms | DP_val=1648 | Gr_val=1610
n= 100 | DP=12.5650ms | Greedy=0.0594ms | DP_val=1754 | Gr_val=1712
n= 200 | DP=28.9229ms | Greedy=0.1189ms | DP_val=1854 | Gr_val=1832
n= 300 | DP=44.9748ms | Greedy=0.1625ms | DP_val=1914 | Gr_val=1860
n= 500 | DP=237.0689ms | Greedy=0.7331ms | DP_val=2002 | Gr_val=1928
n= 700 | DP=221.6877ms | Greedy=0.4764ms | DP_val=2000 | Gr_val=1930
n= 1000 | DP=140.1985ms | Greedy=0.4840ms | DP_val=2061 | Gr_val=1985

Generating charts...
Saved: /content/chart_runtime.png
Saved: /content/chart_quality.png
Saved: /content/chart_theory.png
```

Case	Patients (n)	Capacity (W)	DP Value	Greedy Value	Greedy %
Best Case (small n, high BP)	10	50	150	140	93.3%
Average Case (medium n, mixed)	50	200	670	610	91.0%
Worst Case (large n, tight cap)	200	100	410	390	95.1%